

Fiche récap, journée 1 : Du neurone à la régression logistique

jeudi 8 octobre 2026

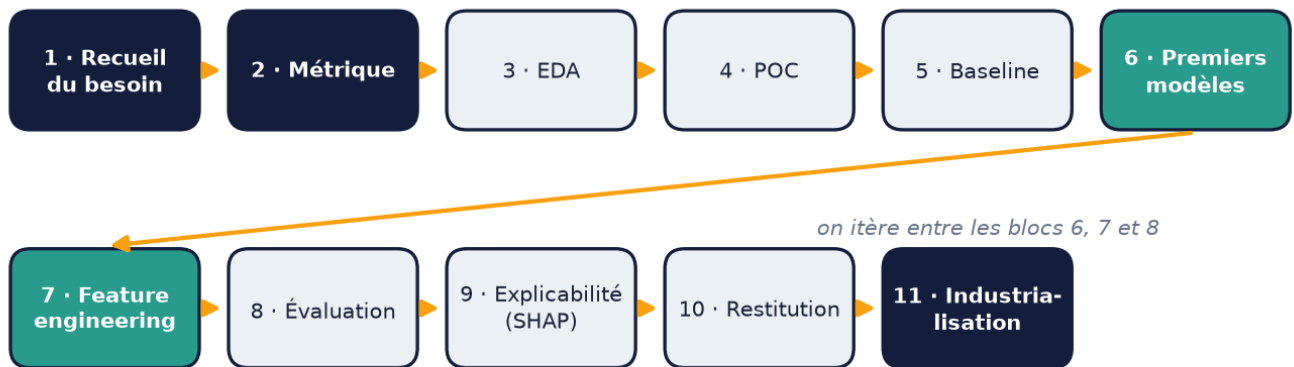
La journée commence par le cadre : supervisé ou non supervisé, les onze blocs d'un projet de machine learning, la métrique, et l'idée qu'un modèle est une fonction de perte minimisée par la descente de gradient. Elle pose ensuite les quatre briques de tout apprentissage (une fonction à régler, une mesure d'erreur, une règle d'amélioration, la vitesse) et les applique au plus petit réseau possible : la régression logistique, écrite en NumPy sans boucle.

■ L'essentiel à retenir

- Supervisé : chaque exemple vient avec sa réponse y (régression : un nombre ; classification : une catégorie ; séries temporelles). Non supervisé : pas de réponse, on cherche une structure (clustering, réduction de dimension, anomalies, génération). Le projet du cours est une régression sur une série temporelle.
- Un projet de machine learning a onze blocs : recueil du besoin, métrique, EDA, POC, baseline, premiers modèles, feature engineering, évaluation, explicabilité (SHAP), restitution, industrialisation. Le deep learning n'intervient qu'aux blocs 6 et 7.
- La baseline est la règle la plus simple (médiane par groupe, dernière valeur) : tout modèle qui ne la bat pas est inutile. Sur l'agence immobilière : 45 k€ au jugé, 31 k€ pour la baseline, 15 k€ pour le meilleur modèle.
- Deux métriques coexistent : celle que le modèle minimise (écart moyen en k€, MAE en MW) et celle que l'entreprise regarde (délai de vente, euros d'écart sur le marché). Une seule métrique à optimiser, les autres exigences deviennent des contraintes.
- Le cœur d'un modèle n'est pas son architecture mais sa fonction de perte : elle est le GPS (à quelle distance de l'objectif), la descente de gradient est le moteur, les paramètres le volant, les données le carburant, la validation le tableau de bord.
- Pour l'ordinateur, une photo 64×64 en couleur est une liste de 12 288 nombres de 0 à 255 ; l'étiquette est un nombre (1 chat, 0 sinon). Un neurone fait une somme pondérée de ces 12 288 nombres puis l'écrase entre 0 et 1 : une droite pliée. Empilés, les plis dessinent tout.
- Backpropagation sur l'appartement ($x = 50$, $y = 150$, $w = 2$, $b = 10$) : $\hat{y} = 110$, $J = (\hat{y} - y)^2 = 1\,600$; $dJ/d\hat{y} = -80$; $dJ/dw = -4\,000$, $dJ/db = -80$; avec $\alpha = 0,0001$, w passe à 2,4 et l'erreur à 400. La perte mesure, la descente de gradient décide, la backpropagation calcule les pentes.
- Standardiser : $x' = (x - \text{moyenne}) / \text{écart-type}$, colonne par colonne, avec les statistiques du train. Sans cela, surface (m^2), chambres et code postal n'ont pas la même échelle et la descente de gradient zigzague.
- Construire X et Y : une photo = une colonne de 12 288 nombres, m photos côte à côte = X de forme $(12\,288, m)$, Y une ligne de m réponses (1 chat, 0 sinon). Cette disposition permet de calculer les m prédictions en une seule multiplication $Z = w^T X + b$.
- Vectoriser, en clair : SIMD (une instruction pour 8 ou 16 nombres), BLAS (des routines écrites une fois pour toutes, appelées par `np.dot`), GPU (des milliers de petits cœurs). 12 milliards de multiplications : 3 h en boucle Python, 1 min en NumPy, 1 s sur GPU.
- $IA \supset ML \supset DL$: le deep learning apprend les features et la décision ensemble ; le ML classique garde l'avantage sur le tabulaire de petite taille (sur 5 000 lignes, un gradient boosting bat presque toujours un réseau).

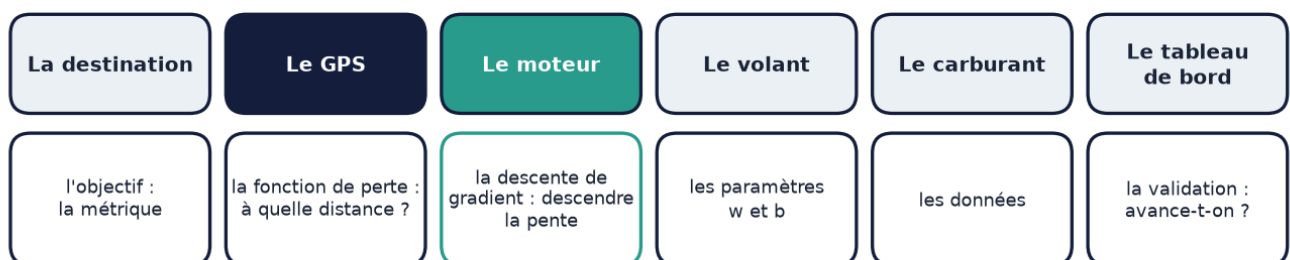
- Apprendre une fonction à partir d'exemples (x, y) demande quatre ingrédients : une famille de fonctions, une mesure d'erreur, une règle pour s'améliorer, et de la vitesse. Les huit journées suivantes ne changent que le premier.
- Cas de l'appartement : 6 ventes, $\text{prix} = w \times \text{surface} + b$. L'essai ($w = 3, b = 20$) donne 45 k€ d'erreur moyenne, l'essai ($w = 3,9, b = 5$) donne 2,3 k€ : l'erreur est le juge, et le signe des écarts dit dans quel sens corriger w .
- Un neurone = combinaison linéaire des entrées + activation (ReLU, sigmoïde...). Il est non linéaire et dérivable : c'est ce qui permet de l'empiler et de l'optimiser par gradient, contrairement à un arbre.
- Un réseau dense empile des neurones ; les couches cachées apprennent leurs propres features. Le réseau $4 \rightarrow 3 \rightarrow 1$ a $4 \times 3 + 3 + 3 \times 1 + 1 = 19$ paramètres.
- Vocabulaire : la perte est ce que l'on minimise, la métrique ce que le métier regarde (F1, MAE) ; une epoch contient des centaines d'itérations ; les paramètres sont appris, les hyperparamètres (α , profondeur, batch) sont choisis.
- Notation du cours : un exemple par colonne. X est de forme (n_x, m) , Y de forme $(1, m)$. Une image $64 \times 64 \times 3$ donne $n_x = 12\,288$; 500 images 32×32 en couleur donnent X de forme $(3072, 500)$.
- Produit matriciel : $(n \times k) \cdot (k \times m) = (n \times m)$, la dimension intérieure doit coïncider. Écrire les dimensions avant de coder ; en NumPy, `@` est le produit matriciel et `*` le produit élément par élément.
- Dérivée = pente locale : bouger a de ϵ fait bouger f de $f'(a) \cdot \epsilon$. La règle de la chaîne multiplie les dérivées locales le long du graphe de calcul, de la sortie vers les entrées : c'est la backpropagation.
- Régression logistique : $\hat{y} = \sigma(w^T x + b) \in (0, 1)$, interprété comme $P(y = 1 | x)$. La décision $\hat{y} \geq 0,5$ équivaut à $z \geq 0$: la frontière est linéaire. $z = 1,5$ donne $\hat{y} = 0,82$.
- La perte logistique découle du maximum de vraisemblance d'une loi de Bernoulli et rend le coût convexe ; l'erreur quadratique donnerait plusieurs minima locaux. Confiant et faux coûte cher : $\hat{y} = 0,1$ pour $y = 1$ donne une perte de 2,3.
- Descente de gradient : $w \leftarrow w - \alpha \cdot dw, b \leftarrow b - \alpha \cdot db$. C'est le seul algorithme d'apprentissage du cours, de la régression logistique à GPT ; pour la régression logistique, $dz = a - y$.
- Le taux d'apprentissage α est le premier hyperparamètre : trop grand, le coût monte ou oscille ; trop petit, il descend très lentement. Ordre de grandeur de départ : 10^{-3} à 10^{-1} selon la normalisation.
- Vectoriser supprime les boucles sur les exemples et sur les features : $Z = w^T X + b, dZ = A - Y, dw = X \cdot dZ^T / m$. Gain $\times 100$ à $\times 300$ sur CPU, bien plus sur GPU.
- PCA : standardiser, centrer, covariance $\Sigma = (1/m) \cdot X \cdot X^T$, vecteurs propres (ou SVD), projeter sur les k premières directions. Choisir k par variance cumulée : sur Fashion-MNIST, ≈ 80 composantes sur 784 pour 90 % de la variance.

■ Les schémas à garder en tête



Les onze blocs d'un projet de machine learning ; le deep learning n'intervient qu'aux blocs 6 et 7.

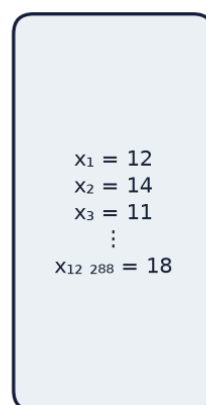
L'architecture est la carrosserie : elle change la forme, pas le principe. Changer de perte, c'est changer de destination.



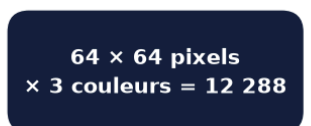
Le cœur d'un modèle : la perte est le GPS, la descente de gradient le moteur, les paramètres le volant, les données le carburant, la validation le tableau de bord.

26	11	13	14	13	26
27	229	231	239	213	18
22	242	200	203	243	24
10	242	213	206	214	20
18	201	240	229	226	24
25	29	25	15	16	22

ce que voit l'ordinateur :
un nombre par pixel (0 noir, 255 blanc)

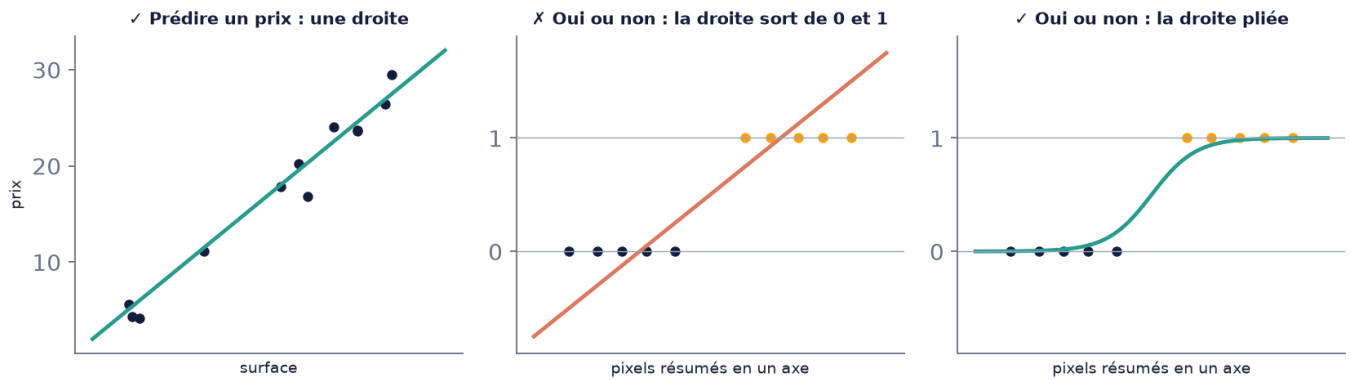


une colonne de
12 288 nombres

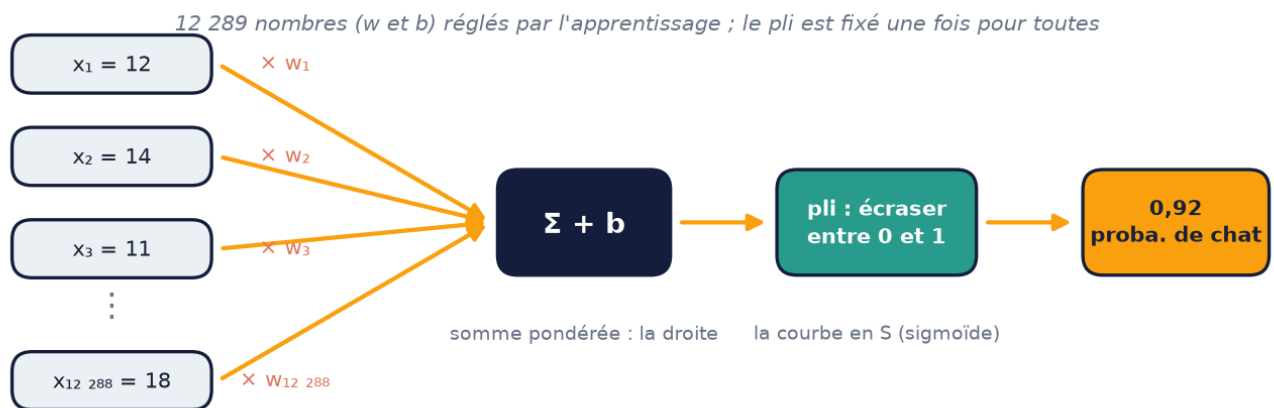


l'entrée du neurone, comme la
surface pour le prix

Pour l'ordinateur, une photo 64 × 64 en couleur est une colonne de 12 288 nombres : l'entrée du neurone, comme la surface pour le prix.



Une droite suffit pour un prix ; pour répondre oui ou non, elle sort de 0 et 1 ; pliée par la sigmoïde, elle devient un neurone.

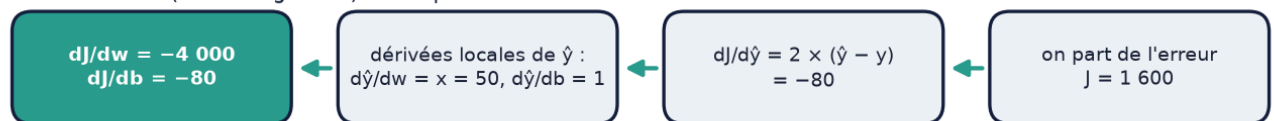


Le neurone de l'intérieur : chaque entrée multipliée par son poids, une somme plus b (la droite), un pli entre 0 et 1 (la courbe en S), une probabilité.

PASSE AVANT (gauche → droite)



PASSE ARRIÈRE (droite → gauche) : multiplier les dérivées locales



Mise à jour ($\alpha = 0,0001$) : $w \leftarrow 2 - 0,0001 \times (-4\ 000) = 2,4$; $b \leftarrow 10 - 0,0001 \times (-80) = 10,008$
nouvelle prédiction $\approx 130\text{ k€}$: l'erreur passe de 1 600 à 400. On recommence.

La backpropagation sur l'appartement : passe avant ($\hat{y} = 110, J = 1\ 600$), passe arrière ($dJ/dw = -4\ 000, dJ/db = -80$), mise à jour ($w = 2,4$, erreur 400).

AVANT : des échelles incomparables

	surface	chambres	code postal
appt A	32	1	33000
appt B	85	3	33800
appt C	140	5	33100
moyenne	86	3	33300
écart-type	44	1,6	350



APRÈS : tout entre -2 et +2

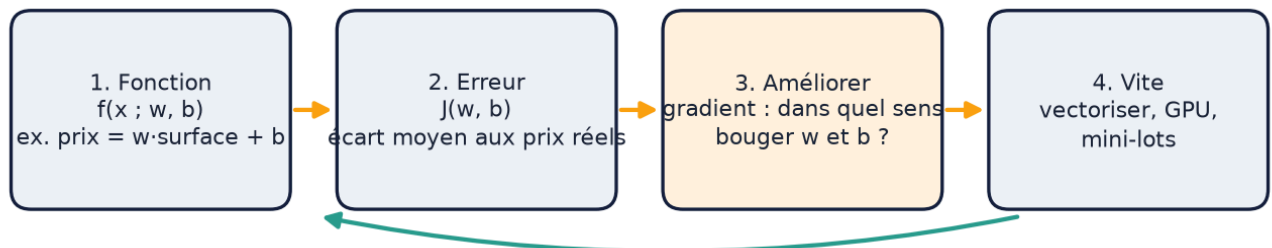
	surface'	chambres'	code postal'
appt A	-1,2	-1,2	-0,9
appt B	0,0	0,0	1,4
appt C	1,2	1,2	-0,6
moyenne	0	0	0
écart-type	1	1	1

statistiques calculées sur le train seulement, jamais sur la sortie y

$$x' = (x - \text{moyenne}) / \text{écart-type}$$

Standardiser : soustraire la moyenne, diviser par l'écart-type, colonne par colonne, avec les statistiques du train ; toutes les entrées parlent la même langue.

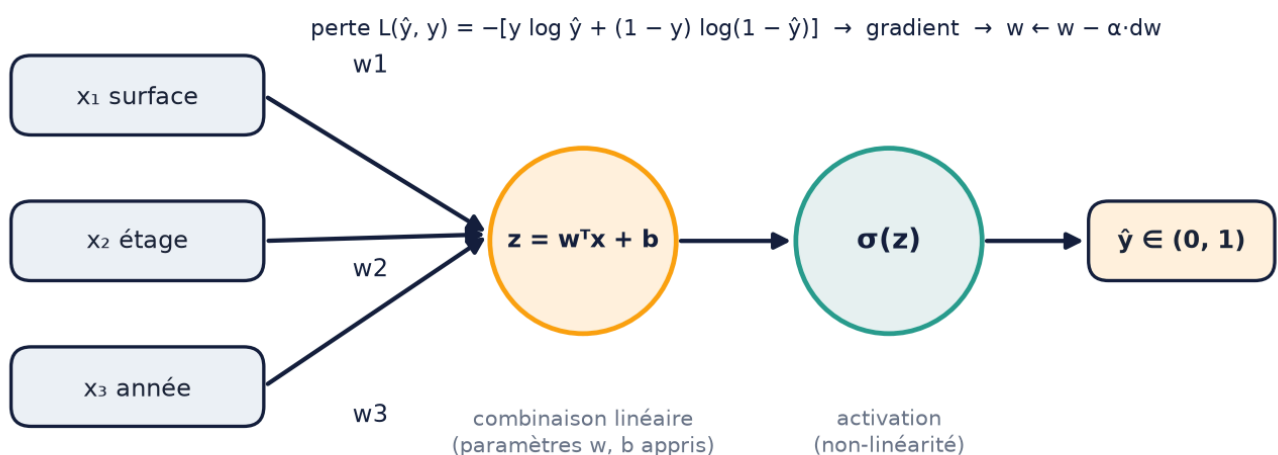
Les quatre ingrédients de l'apprentissage



répéter jusqu'à ce que l'erreur ne baisse plus (descente de gradient)

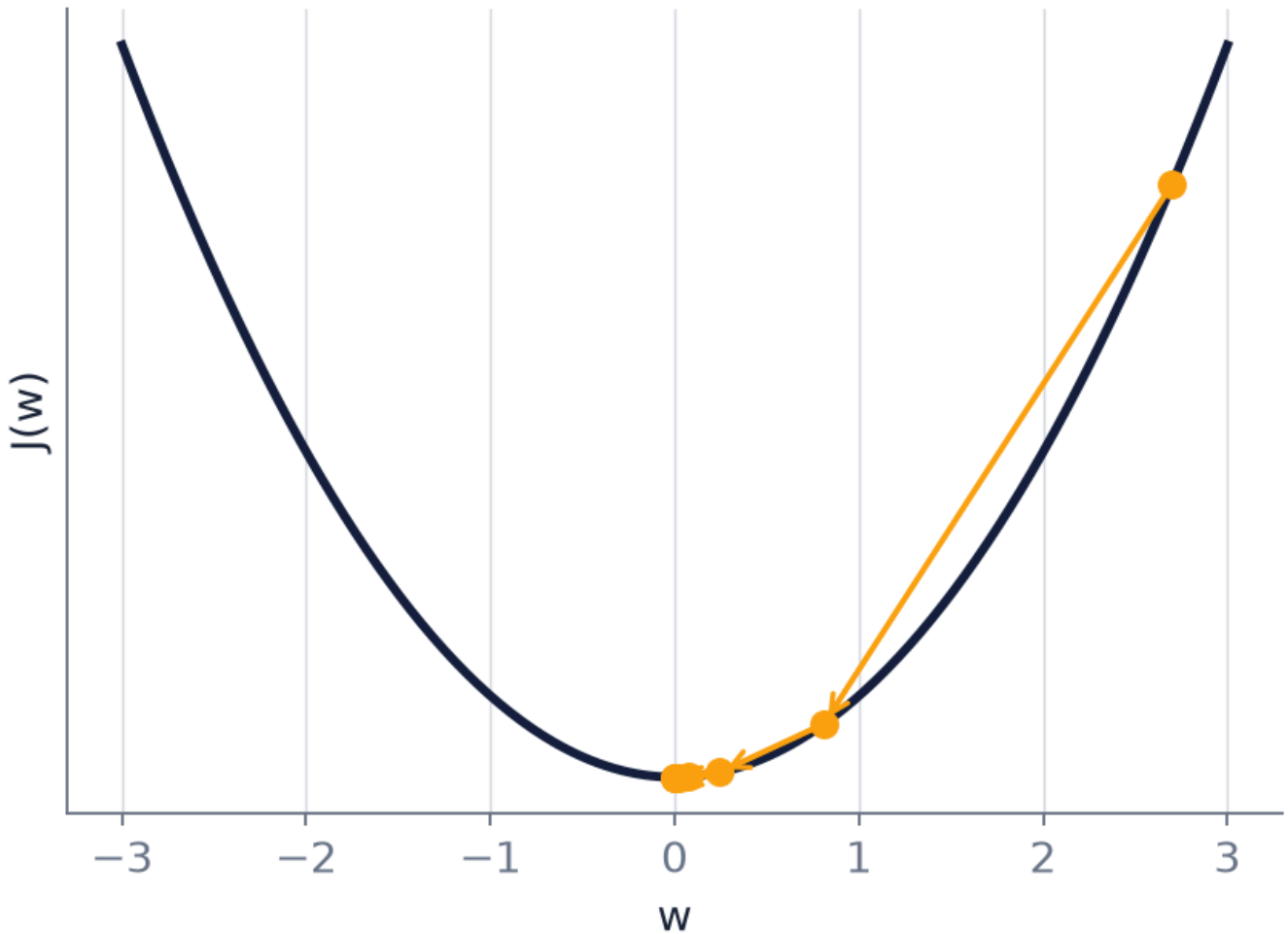
Apprendre, c'est choisir une fonction, mesurer son erreur, l'améliorer par le gradient, et aller vite.

Un neurone = une régression logistique : entrées pondérées, somme, activation



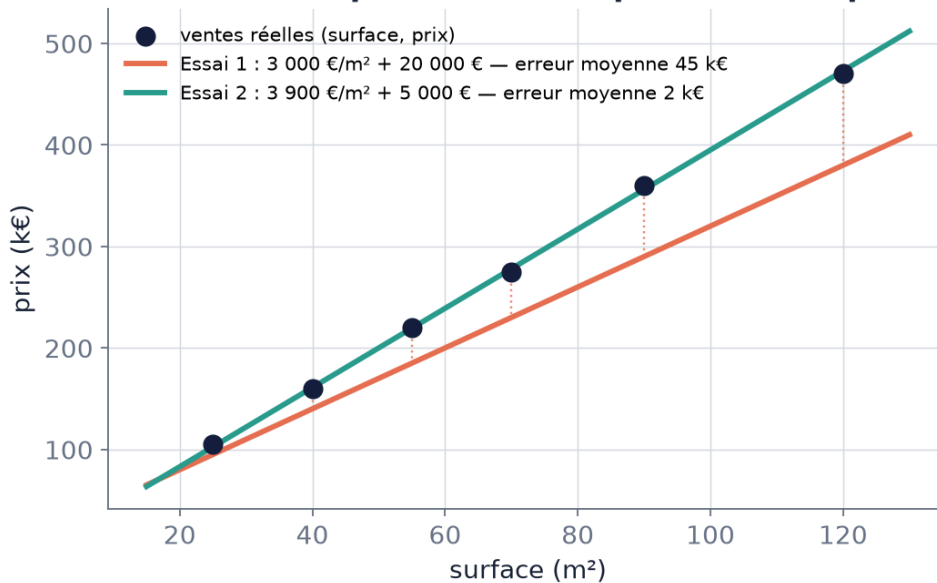
Le neurone : une combinaison linéaire des entrées puis une activation ; les paramètres w et b sont appris.

Descente de gradient ($\alpha = 0.35$)



Descente de gradient : à chaque pas, w bouge dans le sens opposé à la pente, avec un taux α .

Apprendre = choisir w et b qui rendent les pointillés les plus courts possible



Le cas de l'appartement : deux droites candidates, les résidus en pointillés, l'erreur moyenne à réduire.

■ Formules et ordres de grandeur

- $\hat{y} = \sigma(w^T x + b)$ avec $\sigma(z) = 1 / (1 + e^{-z})$; $\sigma(0) = 0,5$; $w \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$.
- Perte $\mathcal{L}(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$; coût J = moyenne des $\mathcal{L}$ sur les m exemples. Un modèle qui prédit 0,5 partout a un coût $\log 2 \approx 0,693$: c'est la valeur de départ typique.
- Passe arrière sur un exemple : $dz = a - y$; $dw = x \cdot dz$; $db = dz$.
- Vectorisé : $Z = w^T X + b$ de forme $(1, m)$; $A = \sigma(Z)$; $dZ = A - Y$; $dw = X \cdot dZ^T / m$ de forme $(n_x, 1)$; $db =$ moyenne de dZ .
- Mise à jour : $w \leftarrow w - \alpha \cdot dw$; $b \leftarrow b - \alpha \cdot db$. dw a toujours la même forme que w .
- Standardisation : $x \leftarrow (x - \mu) / \sigma$, calculées par feature sur le train, puis réutilisées telles quelles sur le test.
- Covariance : $\Sigma = (1/m)(X - \mu)(X - \mu)^T$ de forme (n_x, n_x) ; PCA : $Z = U_k^T \cdot X$ de forme (k, m) , reconstruction $X = U_k \cdot Z$.
- Ordres de grandeur : 12 289 paramètres pour le classifieur de chats ; 10 000 images $\times$ 12 288 pixels $\approx$ 123 millions d'opérations par epoch ($\approx$ 12 s en Python pur, $\approx$ 0,3 s en NumPy).

■ Les pièges classiques

- Le vecteur « rang 1 » de forme $(n,)$: sa transposée ne fait rien et il se diffuse silencieusement en (m, m) au lieu de $(1, m)$. Dans ce cours, tout vecteur est $(n, 1)$ ou $(1, n)$, et l'on écrit `assert w.shape == (n_x, 1)`.
- Confondre `*` (élément par élément, avec broadcasting) et `@` (produit matriciel) : $(3,4) * (4,3)$ lève une erreur, $(3,4) @ (4,3)$ donne $(3,3)$.
- $\log(0) = -\infty$: si A vaut exactement 0 ou 1, le coût devient nan. Clipper A entre $1e-8$ et $1 - 1e-8$ avant le log.
- Oublier le $1/m$ dans dw et db : le gradient est m fois trop grand et la descente diverge pour un α raisonnable.
- Standardiser le jeu de test avec ses propres μ et σ : c'est une fuite de données ; on réutilise ceux du train.
- Lancer une PCA sans standardiser : la feature à grande échelle (le prix en k€ face à la surface en m²) domine la première composante.

■ Pour le projet

- Avant tout modèle, la checklist d'exploration : formes et types, distributions, 20 exemples au hasard, corrélations, baseline, et un découpage train / dev / test fait AVANT toute décision, en respectant l'ordre du temps pour une série.
- La baseline classique est obligatoire : c'est le score à battre, et conclure « la baseline suffit » est acceptable si la comparaison avec plusieurs réseaux est réellement faite.
- Les réflexes de la journée (formes écrites avant le code, `assert` sur les shapes, pas de boucle sur des nombres) s'appliquent à tout le code du projet, en NumPy comme en PyTorch.
- Situer chaque étape du projet sur les onze blocs : besoin (les acheteurs d'énergie), métrique (MAE en MW), EDA et baseline (journée 3), modèles et feature engineering (journées 3 à 6), évaluation (douze plis), restitution (soutenance), industrialisation (journée 4).
- Pour la journée 2 : relire le TP1. Le projet et son jeu de données seront présentés en fin de journée 2 ; les binômes se forment à la journée 3.

■ Les questions Kahoot de la journée, avec les réponses

Partie 1 : un projet de machine learning, et pourquoi le deep learning

À jouer vers 10h40, après la section « pourquoi le deep learning », avant la pause. 12 questions, 10 minutes.

1. Regrouper 12 000 clients en cinq familles sans aucune étiquette : de quel type d'apprentissage s'agit-il ?

- A. Supervisé : une régression
- B. Supervisé : une classification
- C. Non supervisé : un clustering ✓
- D. Auto-supervisé

Pas de réponse y connue : on cherche une structure dans les x. Clustering, réduction de dimension et anomalies sont non supervisés.

2. Prévoir la consommation électrique de demain à partir de l'historique : quel type de problème ?

- A. Classification binaire
- B. Régression sur une série temporelle ✓
- C. Clustering
- D. Réduction de dimension

La réponse est un nombre (des MW) connu dans les données passées : supervisé, régression ; la dimension temps en fait une série temporelle.

3. Dans les onze blocs d'un projet, lequel fixe le plancher que tout modèle doit battre ?

- A. Le recueil du besoin
- B. La preuve de concept
- C. La baseline ✓
- D. L'évaluation

La règle la plus simple (médiane par quartier × surface : 31 k€) : un modèle qui ne la bat pas est inutile.

4. Sur les onze blocs, où le deep learning intervient-il ?

- A. Partout, c'est une autre façon de faire un projet
- B. Seulement aux blocs modèles et feature engineering ✓
- C. Seulement à l'industrialisation
- D. Au recueil du besoin et à la métrique

Le réseau est un modèle parmi d'autres qui apprend une partie de ses variables ; les neuf autres blocs sont identiques à un projet classique.

5. L'agence immobilière mesure « part des biens vendus en moins de 60 jours ». C'est quoi ?

- A. La métrique d'apprentissage
- B. La métrique métier ✓

- C. Une contrainte
- D. La fonction de perte

Ce que l'entreprise regarde ; le modèle, lui, minimise l'écart moyen en k€. Le travail du data scientist est de relier les deux.

6. À quoi sert SHAP dans un projet ?

- A. À accélérer l'entraînement
- B. À expliquer chaque prédiction par la contribution de chaque variable ✓
- C. À choisir le taux d'apprentissage
- D. À déployer le modèle

Vérifier que le modèle apprend quelque chose de sensé, le déboguer (fuites) et le faire accepter : 312 k€ = moyenne + 85 (surface) – 12 (DPE F) – 9 (étage).

7. Dans la comparaison avec la voiture, quel est le rôle de la fonction de perte ?

- A. Le moteur : elle fait avancer
- B. Le GPS : elle dit à quelle distance on est de l'objectif ✓
- C. Le volant : ce qu'on règle
- D. Le carburant

La perte mesure l'écart à l'objectif pour les paramètres actuels ; la descente de gradient est le moteur, les paramètres le volant, les données le carburant.

8. Appartements, essai 1 (prix = 3 × surface + 20) : prédictions trop basses, surtout les grandes. Que faire ?

- A. Augmenter w, le prix au m² ✓
- B. Diminuer w, le prix au m²
- C. Changer de famille de fonctions
- D. Ajouter des ventes au jeu de données

Les écarts grandissent avec la surface : monter w (3,9 au lieu de 3) fait passer l'erreur moyenne de 45 à 2 k€.

9. Une photo 64 × 64 en couleur, pour l'ordinateur, c'est combien de nombres ?

- A. 64
- B. 4 096
- C. 12 288 ✓
- D. Trois : rouge, vert, bleu

64 × 64 pixels × 3 couleurs = 12 288 nombres de 0 à 255 : c'est l'entrée du neurone, comme la surface pour le prix.

10. Pourquoi une simple droite ne suffit-elle pas pour répondre « chat ou pas chat » ?

- A. Elle est trop lente à calculer
- B. Elle sort de 0 et 1 : impossible de la lire comme une probabilité ✓
- C. Elle a trop de paramètres
- D. Elle ne marche que pour les images

Une droite donnerait 1,8 ou -0,4 ; pliée entre 0 et 1 par la sigmoïde, elle devient un neurone et donne 0,98 ou 0,02.

11. Qu'est-ce qu'un neurone, au sens de ce cours ?

- A. Une fonction d'activation (ReLU) appliquée à l'entrée
- B. Un arbre de décision rendu dérivable
- C. Une moyenne des sorties des neurones précédents
- D. Une somme pondérée des entrées suivie d'une activation ✓

$z = w \cdot x + b$ puis $a = g(z)$: la partie linéaire est apprise, l'activation apporte le pli ; empilés, les plis dessinent n'importe quelle forme.

12. Jeu tabulaire de 5 000 lignes et 30 colonnes : sur quel modèle parier en premier ?

- A. Un réseau dense à 10 couches
- B. Un Transformer
- C. Un gradient boosting ✓
- D. Un CNN

En régime petites données tabulaires, le boosting bat presque toujours un réseau ; c'est la baseline à battre.

Partie 2 : maths, régression logistique, vectorisation et ACP

À jouer en fin de matinée, après la régression logistique, puis compléter en fin de journée. 13 questions, 10 minutes.

1. 500 images 32×32 en couleur. Quelle est la forme de X dans la notation du cours ?

- A. (3072, 500) ✓
- B. (500, 3072)
- C. (1024, 500)
- D. (32, 32, 3, 500)

Un exemple par colonne : $n_x = 32 \times 32 \times 3 = 3072$ lignes et $m = 500$ colonnes.

2. $J = 3 \cdot (a + b \cdot c)$ avec $a = 5$, $b = 3$, $c = 2$. Que vaut dJ/dc ?

- A. 3
- B. 6
- C. 33
- D. 9 ✓

Règle de la chaîne : $dJ/dc = dJ/dv \cdot dv/du \cdot du/dc = 3 \times 1 \times b = 9$.

3. Appartement : $x = 50$, $y = 150$, $w = 2$, $b = 10$, $J = (\hat{y} - y)^2$. Que vaut dJ/dw ?

- A. -40
- B. -80
- C. -4 000 ✓
- D. +4 000

$\hat{y} = 110$, $dJ/d\hat{y} = 2 \times (110 - 150) = -80$, $d\hat{y}/dw = x = 50$: $dJ/dw = -80 \times 50 = -4\,000$. Négatif : il faut augmenter w .

4. La perte mesure, la descente de gradient décide... et la backpropagation ?

- A. Choisit le taux d'apprentissage
- B. Calcule toutes les pentes dJ/dw en une passe arrière ✓
- C. Met à jour les poids
- D. Mesure l'erreur sur le test

Sans elle, il faudrait 12 289 passes avant pour une seule mise à jour ; avec elle, une passe avant et une passe arrière suffisent.

5. Pourquoi la perte logistique plutôt que l'erreur quadratique pour la régression logistique ?

- A. Elle est plus rapide à calculer
- B. Le coût devient convexe et elle découle du maximum de vraisemblance ✓
- C. Elle borne la sortie entre 0 et 1
- D. Elle dispense de standardiser les données

Avec l'erreur quadratique le coût a plusieurs minima locaux ; la log-loss est convexe et probabiliste.

6. Un client a résilié ($y = 1$). Le modèle avait prédit $\hat{y} = 0,1$. Quelle est la perte sur cet exemple ?

- A. 0,1
- B. 0,105
- C. 0,9
- D. 2,3 ✓

$-\log 0,1 \approx 2,3$, contre 0,105 si le modèle avait prédit 0,9 : être confiant et faux coûte cher.

7. Relecture : pour la régression logistique, que vaut $dz = \partial \mathcal{L} / \partial z$?

- A. $a - y$ ✓
- B. $y - a$
- C. $a(1 - a)$
- D. $x \cdot (a - y)$

$da \cdot a(1 - a)$ se simplifie en $a - y$: l'erreur de prédiction est le signal qui corrige les poids.

8. Le coût augmente d'une itération à l'autre pendant la descente de gradient. Que suspecter en premier ?

- A. α trop grand ✓
- B. α trop petit
- C. Trop d'epochs
- D. Pas assez de features

Un pas trop long dépasse le minimum et s'en éloigne ; sur l'appartement, $\alpha = 0,01$ fait sauter w de 2 à 42.

9. Appartement A : surface 32 m², moyenne 86, écart-type 44. Sa surface standardisée ?

- A. -54
- B. -1,2 ✓
- C. 0,37
- D. 1,2

$(32 - 86) / 44 = -1,2$: un pas de 1 vaut désormais un écart-type, pour toutes les colonnes.

10. Comment standardiser le jeu de test avant de l'envoyer au modèle ?

- A. Avec les μ et σ calculés sur le jeu d'entraînement ✓
- B. Avec sa propre moyenne et son propre écart-type
- C. On ne standardise jamais le test
- D. Avec $\mu = 0$ et $\sigma = 1$ par convention

En production le modèle ne connaît que les statistiques du train ; recalculer sur le test est une fuite.

11. Produit scalaire sur un million d'éléments : gain typique de np.dot sur une boucle Python, sur CPU ?

- A. $\times 2$ à $\times 5$
- B. $\times 10$
- C. Aucun : c'est le même calcul
- D. $\times 100$ à $\times 300$ ✓

≈ 300 ms en boucle contre ≈ 1 ms avec la routine BLAS compilée (SIMD) ; un GPU va encore plus loin.

12. $a = \text{np.random.randn}(5)$. Que vaut $a.T.\text{shape}$?

- A. (5,) ✓
- B. (1, 5)
- C. (5, 1)
- D. Une erreur

Un vecteur « rang 1 » n'est ni ligne ni colonne : la transposée ne fait rien. Toujours créer (5, 1) ou (1, 5).

13. Fashion-MNIST, 784 pixels : combien de composantes principales pour garder 90 % de la variance ?

- A. 2

- B. 10
- C. Environ 80 ✓
- D. Environ 700

2 composantes portent $\approx 47\%$, 10 en portent $\approx 72\%$, il en faut ≈ 80 pour 90% et ≈ 190 pour 95% .